OPEN

GitHub Copilot Business fundamentals

- A governance-aware path from assistance to accountable delivery
- Facilitator release 1.0.0 | 2026-10-14

Audience, agenda, and learning rhythm

- Developers, technical leads, administrators, security partners, and adoption owners
- Learn a durable concept, practise it in the repository, then capture evidence
- Allow about 3 hours plus optional cloud-agent time

Seven outcomes, one evidence trail

- Establish the managed boundary and select verifiable context
- Use completion, chat, edit, and agent workflows deliberately
- Delegate responsibly, customize with least privilege, and measure adoption carefully

Prepare the learning environment

- Use a current VS Code client, the Copilot extension, and the licensed GitHub account
- Clone this repository and install dependencies before the session
- Use fictitious data only; the grant prototype remains local and unofficial

Sources: E07

/lab/en#lab-00

Plan context: capability follows managed access

- Business centers organization access, policy, and reporting
- Enterprise requires GitHub Enterprise Cloud and includes Business capabilities
- Copilot is not currently available for GitHub Enterprise Server; recheck before delivery

Sources: E01, E02, E03, E05

Start with a managed trust boundary

- Confirm the seat, organization policy, approved surfaces, and repository permission
- Use the smallest useful file and tool scope
- Keep secrets, credentials, and real applicant data outside prompts and history

Treat every output as a hypothesis

- Check correctness, security, licensing, privacy, accessibility, and organizational standards
- Ask for evidence, inspect the source, and run the narrowest useful validation
- Human approval remains at commands, workflows, merges, and deployment boundaries

Sources: E41, E42

/lab/en#lab-01

Prompt for a verifiable result

- Goal: name the observable change
- Context: point to exact files, symbols, constraints, and examples
- Evidence: request tests, diagnostics, diffs, and stated uncertainty

Choose the least autonomous useful mode

- Completion for local continuation; chat for explanation and options
- Edit for scoped changes; agent mode for iterative multi-step work
- Cloud agent for bounded asynchronous work reviewed through a pull request

Sources: E08, E09

Completions: keep the acceptance decision local

- Write intent in nearby code, types, tests, and names
- Accept only the useful portion and reject plausible-looking drift
- Run the focused test before expanding the change

/lab/en#lab-02

Chat: explain, compare, then verify

- Select the smallest code surface that can answer the question
- Ask for assumptions, alternatives, and a falsifiable check
- Compare the response with code, documentation, and runtime evidence

/lab/en#lab-02

Edit workflow: scope before applying

- State files that may change and behavior that must remain stable
- Review each hunk; do not accept unrelated cleanup
- Compile or test the touched slice immediately

/lab/en#lab-03

Agent mode: grant tools one boundary at a time

- Describe the outcome, constraints, and stop conditions
- Review proposed commands and watch for scope expansion
- Require a clean diff and executable evidence before completion

Sources: E08, E42

/lab/en#lab-06

Cloud delegation: define a reviewable task

- Delegate bounded backlog work with acceptance criteria and repository context
- As of 2026-10-14, each task is limited to one repository, branch, pull request, and 59-minute session
- Never let asynchronous execution bypass required review or deployment gates

Sources: E09, E10, E11

/lab/en#lab-07

Review loop: evidence before merge

- Read the task framing, session log, commits, and full diff
- Run tests and security checks independently of the agent summary
- Record the human judgment that accepted, corrected, or rejected the work

Sources: E11, E42

/lab/en#lab-07

Customization is a layered contract

- Instructions set automatic standards; prompt files package repeatable requests
- Custom agents specialize tools and context; skills load relevant procedures on demand
- Hooks run deterministic lifecycle commands; MCP connects approved external capabilities

Sources: E16, E17, E18

/lab/en#lab-06

Instructions and prompt files make intent reviewable

- Keep organization-independent standards close to the repository
- Use path-specific instructions only where their scope is clear
- Write prompts with inputs, output shape, validation, and stop conditions

Sources: E16

/lab/en#lab-06

Skills and hooks separate judgment from determinism

- Use a skill for reusable context, process, scripts, and resources
- Use a hook when a command must run predictably at a lifecycle event
- Review both as executable supply-chain inputs

Sources: E16, E17

/lab/en#lab-06

MCP: useful reach requires least privilege

- Approve servers, tools, data sources, and authentication paths explicitly
- Managed MCP policy scope and defaults depend on how Business or Enterprise access is provided
- Treat returned content and tool instructions as untrusted until reviewed

Sources: E18, E19

/lab/en#lab-06

Model choice: fit, policy, cost, then validation

- Auto is dynamic selection, not a fixed model alias
- Visible models still depend on plan, policy, client, rollout, and provider-specific terms
- Organization permission governs model switching and bring-your-own-key use

Sources: E12, E13, E14, E15

Administration sets the usable boundary

- Billing begins when access is granted; mid-cycle grants are prorated
- Assignment workflows and enterprise policy scope vary by plan, access source, and delegated control
- Govern agents, models, AI-credit budgets, privacy, features, and MCP without assuming uniform defaults

Sources: E19, E20, E30, E36, E37, E38, E40

Content exclusion is a partial control

- Coverage varies by feature and client; editor Edit and Agent modes have limitations
- Excluded semantics can still be exposed indirectly by allowed content
- Keep access control, secret management, scanning, and developer judgment in place

Sources: E23, E39

/lab/en#lab-05

Metrics guide questions, not causal claims

- Coverage includes selected IDE, CLI, and agent activity, not every Copilot surface
- Allow two full UTC days and verify organization attribution and current client telemetry
- The 2026-10-06 incident confirms that affected historical agent activity cannot be backfilled

Sources: E20, E21, E22, E31

/lab/en#lab-07

LAB

Lab 00 | Establish the managed trust boundary

- Inspect the local-only data boundary and permanent safety notice
- Run the focused bilingual foundation tests
- Checkpoint: state safe context and prohibited data

/lab/en#lab-00

LAB

Lab 01 | Give Copilot verifiable repository context

- Select exact files and ask for a source-grounded explanation
- Separate observed facts from assumptions
- Checkpoint: every material statement has a code or documentation anchor

/lab/en#lab-01

LAB

Lab 02 | Use completion and chat for focused tests

- Use nearby types and tests to shape a bounded suggestion
- Compare chat guidance with the real implementation
- Checkpoint: focused tests pass and unsupported changes are rejected

/lab/en#lab-02

LAB

Lab 03 | Plan and validate a small refactoring

- Define invariants and allowed files before editing
- Review the plan and each applied hunk
- Checkpoint: behavior remains stable under focused validation

/lab/en#lab-03

LAB

Lab 04 | Generate an accessible browser test

- Ask for keyboard, semantic, and narrow-viewport evidence
- Inspect selectors for user-facing meaning rather than implementation detail
- Checkpoint: the test fails for the intended defect and passes after correction

/lab/en#lab-04

LAB

Lab 05 | Review privacy and local file behaviour

- Trace PDF selection and IndexedDB behavior without real applicant data
- Challenge overbroad claims about exclusion or retention
- Checkpoint: privacy statements match code and current first-party terms

Sources: E24, E25, E26

/lab/en#lab-05

LAB

Lab 06 | Use agent mode with explicit guardrails

- Set file, command, and stop boundaries before execution
- Approve tools incrementally and inspect every change
- Checkpoint: evidence is complete without scope expansion

Sources: E08, E42

/lab/en#lab-06

LAB

Lab 07 | Delegate, review, and measure responsibly

- Use the mock walkthrough when cloud agent access is unavailable
- Review task framing, session evidence, pull-request changes, and checks
- Checkpoint: propose one enablement action and one guardrail from the evidence

Sources: E09, E11, E22

/lab/en#lab-07

LAB

Capstone | From prompt to reviewed evidence

- Frame one small issue with constraints and acceptance criteria
- Produce a reviewed change, tests, security checks, and relevant customization
- Reflect on what required human judgment and what evidence supports completion

/lab/en#lab-cap

Dated appendix | Plans and commercial context (2026-10-14)

- Business: $19 USD per granted seat per month; 1,900 AI credits per user per month
- Enterprise: $39 USD per granted seat per month; 3,900 credits; pooled allowances do not roll over
- Additional usage: $0.01 USD per token-priced AI credit; paid completions remain unlimited; verify the purchasing channel

Sources: E03, E04, E40

https://docs.github.com/en/copilot/get-started/plans

Dated appendix | Feature and model volatility (2026-10-14)

- IDE support, model availability, previews, and cloud-agent limits can change
- Use live feature, model, retirement, and provider-footnote references on delivery day
- No exact retention duration is asserted; verify the service, purchasing channel, provider, and contract

Sources: E06, E10, E12, E26, E30, E32

https://docs.github.com/en/copilot/reference/copilot-feature-matrix

References and freshness ownership

- Course owner: plans, capabilities, and preview status before every release
- Security and privacy owners: exclusions, agents, terms, Trust Center, and retention before delivery
- Adoption owner: metrics coverage, lag, attribution, and client requirements before reporting

Sources: E20, E21, E22, E26, E31

https://docs.github.com/en/copilot